

WEEK – 08

Implement Johnson Trotter algorithm to generate permutations.

Pseudocode:

Initialize array $\text{perm}[1..n] = \{1, 2, \dots, n\}$

Initialize direction $\text{dir}[1..n] = \text{LEFT}$ for all

Print perm

Repeat:

 mobile = -1

 pos = -1

 // Find largest mobile element

 For i = 1 to n:

 If $\text{dir}[i] == \text{LEFT}$ and $i > 1$ and $\text{perm}[i] > \text{perm}[i-1]$:

 If $\text{perm}[i] > \text{mobile}$:

 mobile = $\text{perm}[i]$

 pos = i

 If $\text{dir}[i] == \text{RIGHT}$ and $i < n$ and $\text{perm}[i] > \text{perm}[i+1]$:

 If $\text{perm}[i] > \text{mobile}$:

 mobile = $\text{perm}[i]$

 pos = i

 If mobile == -1:

 Stop

 // Swap in its direction

If dir[pos] == LEFT:

 swap perm[pos] and perm[pos-1]

 swap dir[pos] and dir[pos-1]

 pos = pos - 1

Else:

 swap perm[pos] and perm[pos+1]

 swap dir[pos] and dir[pos+1]

 pos = pos + 1

// Reverse direction of larger elements

For i = 1 to n:

 If perm[i] > mobile:

 dir[i] = opposite(dir[i])

Print perm

Until done

Program:

```
#include <stdio.h>
```

```
#define LEFT -1
```

```
#define RIGHT 1
```

```
void printPermutation(int perm[], int dir[], int n) {
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (dir[i] == LEFT)
```

```
            printf("<-%d ", perm[i]);
```

```
        else
            printf("%d-> ", perm[i]);
    }
    printf("\n");
}
```

```
int getMobile(int perm[], int dir[], int n) {
    int mobile = -1, index = -1;

    for (int i = 0; i < n; i++) {
        if (dir[i] == LEFT && i > 0 && perm[i] > perm[i - 1]) {
            if (perm[i] > mobile) {
                mobile = perm[i];
                index = i;
            }
        }
        if (dir[i] == RIGHT && i < n - 1 && perm[i] > perm[i + 1]) {
            if (perm[i] > mobile) {
                mobile = perm[i];
                index = i;
            }
        }
    }
    return index;
}
```

```
void johnsonTrotter(int n) {
    int perm[n], dir[n];
```

```
for (int i = 0; i < n; i++) {
```

```
    perm[i] = i + 1;
```

```
    dir[i] = LEFT;
```

```
}
```

```
printPermutation(perm, dir, n);
```

```
while (1) {
```

```
    int mobileIndex = getMobile(perm, dir, n);
```

```
    if (mobileIndex == -1)
```

```
        break;
```

```
    int swapIndex = mobileIndex + dir[mobileIndex];
```

```
    printf("Swap %d and %d\n", perm[mobileIndex], perm[swapIndex]);
```

```
    int temp = perm[mobileIndex];
```

```
    perm[mobileIndex] = perm[swapIndex];
```

```
    perm[swapIndex] = temp;
```

```
    temp = dir[mobileIndex];
```

```
    dir[mobileIndex] = dir[swapIndex];
```

```
    dir[swapIndex] = temp;
```

```
    mobileIndex = swapIndex;
```

```
    int mobileValue = perm[mobileIndex];
```

```

        for (int i = 0; i < n; i++) {
            if (perm[i] > mobileValue)
                dir[i] = -dir[i];
        }

        printPermutation(perm, dir, n);
    }
}

```

```

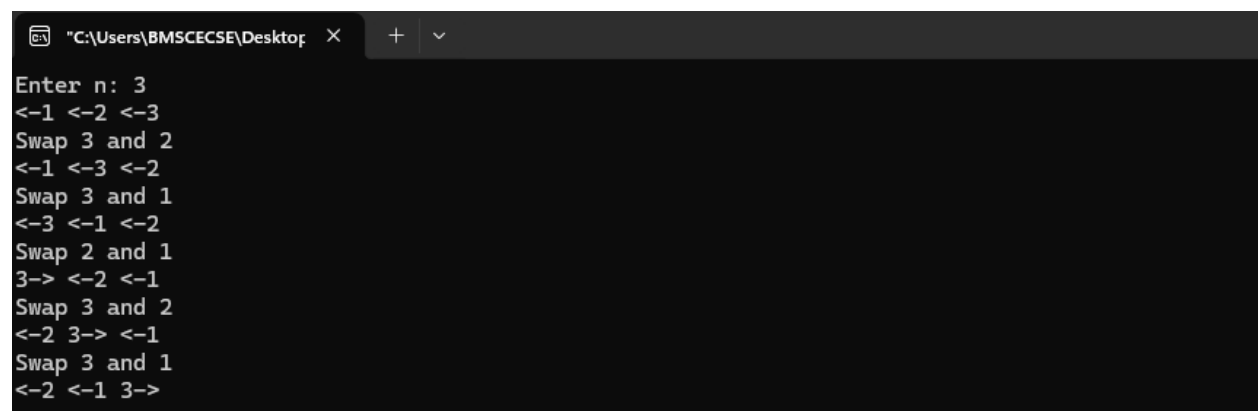
int main() {
    int n;
    printf("Enter N: ");
    scanf("%d", &n);

    johnsonTrotter(n);

    return 0;
}

```

Output:



```

Enter n: 3
<-1 <-2 <-3
Swap 3 and 2
<-1 <-3 <-2
Swap 3 and 1
<-3 <-1 <-2
Swap 2 and 1
3-> <-2 <-1
Swap 3 and 2
<-2 3-> <-1
Swap 3 and 1
<-2 <-1 3->

```